POCKET OPS | MVP MOBILE | APPSCRIPT + FLUTTER

PO

MANUAL OPERACIONAL

Manual de Instalação do Backend AppScript para o MVP Mobile

Guia prático para criar, copiar, configurar e publicar exatamente o backend em Google Apps Script que o app Flutter entregue em **mobile_app/** espera consumir.

VERSÃO	ESCOPO	DATA	BASE
Pocket Ops v2.0.0	**MVP Mobile + integração real**	**27/03/2026**	**Entrega já criada no repositório**

Este documento não inventa camadas novas. Ele espelha somente os arquivos, endpoints, parâmetros e ajustes realmente necessários para a integração Flutter + AppScript já entregue.

Esquilo Invest • Instalação backend para operação local

1. VISÃO GERAL

O que este manual cobre

FLUXO REAL ENTREGUE

Este guia mostra como deixar o AppScript pronto para responder ao app Flutter em produção local, sem alterar a regra de negócio e sem criar backend paralelo.

Papel do AppScript

Servir HTML do dashboard atual e JSON para o app mobile no mesmo deploy.

O que será configurado

Arquivos .gs, um arquivo .html, Script Properties, Web App e URL final /exec.

Resultado esperado

Flutter consumindo `?format=json&resource=...` com o contrato já entregue.

Caminho mais seguro

Atualizar o projeto AppScript existente do repositório, sem renomear arquivos.

Resumo funcional da integração

- O Flutter lê a URL base do backend por **APP_SCRIPT_BASE_URL**.
- O token opcional é lido por **APP_SCRIPT_API_TOKEN**.
- O AppScript responde JSON quando recebe **format=json**.
- Os recursos mobile entregues são **dashboard**, **ai-analysis** e **health**.
- O app mobile não recria lógica de negócio; ele consome o mesmo **getDashboardData()**.

> Cenário recomendado: manter o projeto AppScript já existente e apenas aplicar os arquivos e ajustes listados neste manual. Recriar do zero é possível, mas exige copiar os arquivos de suporte também.

Resultado esperado ao final

- A URL **/exec?format=json&resource=health** responde com **ok: true**.
- A URL **/exec?format=json&resource=dashboard** devolve o payload usado na home mobile.
- O Flutter deixa de mostrar o estado de configuração e passa a carregar patrimônio, score e radar.
- A leitura da Esquilo IA fica disponível sob demanda quando as chaves da IA estiverem configuradas.

2. PRÉ-REQUISITOS

O que precisa existir antes de começar

CHECKLIST DE ENTRADA

ACESSOS

Acesso de edição ao projeto Google Apps Script, à planilha operacional e permissão para implantar Web App.

ARQUIVOS FONTE

Este repositório local com as pastas **apps_script/**, **frontend/html/** e **mobile_app/**.

CONFIGURAÇÃO FLUTTER

Capacidade de executar o app com **--dart-define=APP_SCRIPT_BASE_URL=...**.

FONTE DE DADOS

A planilha indicada em **apps_script/utils/Config.gs** ou a infraestrutura BigQuery já existente.

PERMISSÕES GOOGLE

Conta Google com autorização para usar Apps Script, Spreadsheet e, se mantiver a arquitetura atual, BigQuery.

CHAVES OPCIONAIS

MOBILE_APP_API_TOKEN para proteger o acesso externo e **GEMINI_API_KEY** para a Esquilo IA.

> Se você quer apenas subir a home mobile e o radar da carteira, o bloco mais urgente é: **Config.gs + Backend_Core.gs + Mobile_Api.gs + deploy /exec**. Os demais arquivos entram para manter a solução entregue funcionando sem desvio de arquitetura.

3. ESTRUTURA DOS ARQUIVOS

Arquivos do AppScript que entram nesta instalação

FORMATO EXATO

A tabela abaixo separa o que você precisa criar ou editar primeiro, o que precisa existir como suporte e o que fica fora do escopo do MVP mobile atual.

Arquivos para criar ou editar primeiro

NOME NO APPSCRIPT	TIPO	ORIGEM NO REPOSITÓRIO	FINALIDADE	STATUS
Config.gs	Script .gs	`apps_script/utils/Config.gs`	Configura versão, spreadsheetId, nomes de abas e chaves de Script Properties.	OBRIGATÓRIO
Backend_Core.gs	Script .gs	`apps_script/backend/Backend_Core.gs`	Serve o HTML atual e também redireciona chamadas mobile para o envelope JSON.	OBRIGATÓRIO
Mobile_Api.gs	Script .gs	`apps_script/backend/Mobile_Api.gs`	Expõe os recursos **dashboard**, **ai-analysis** e **health**.	OBRIGATÓRIO
Dashboard.html	HTML .html	`frontend/html/Dashboard.html`	Mantém o deploy web atual funcional quando a URL é aberta sem **format=json**.	RECOMENDADO

Arquivos de suporte para refletir exatamente a solução entregue

NOME NO APPSCRIPT	TIPO	ORIGEM NO REPOSITÓRIO	FINALIDADE	STATUS
Sheet_Readers.gs	Script .gs	`apps_script/services/Sheet_Readers.gs`	Fallback da planilha e normalização dos registros.	OBRIGATÓRIO
Portfolio_Metrics.gs	Script .gs	`apps_script/services/Portfolio_Metrics.gs`	Resumo consolidado, snapshots, alocação e métricas que a	OBRIGATÓRIO

NOME NO APPSCRIPT	TIPO	ORIGEM NO REPOSITÓRIO	FINALIDADE	STATUS
			home mobile exibe.	
Decision_Engine.gs	Script .gs	`apps_script/services/Decision_Engine.gs`	Score, radar, recomendações e leitura operacional da carteira.	OBRIGATÓRIO
BigQueryService.gs	Script .gs	`apps_script/services/BigQueryService.gs`	Fonte principal de dados da arquitetura atual, com consulta às tabelas operacionais.	RECOMENDADO
AI_Service.gs	Script .gs	`apps_script/services/AI_Service.gs`	Entrega a leitura da Esquilo IA para o recurso **ai-analysis**.	SÓ PARA IA
Api_Keys.gs	Script .gs	`apps_script/integrations/Api_Keys.gs`	Leitura das chaves da IA por Script Properties.	SÓ PARA IA

Arquivos fora do escopo do MVP mobile atual

NOME	TIPO	ORIGEM NO REPOSITÓRIO	OBSERVAÇÃO	STATUS
Operational_CRUD.gs	Script .gs	`apps_script/backend/Operational_CRUD.gs`	Operações de cadastro/edição não são necessárias para a primeira entrega mobile.	FORA DO ESCOPO
Export_Import.gs	Script .gs	`apps_script/backend/Export_Import.gs`	Não é usado pelo fluxo atual do app Flutter entregue.	FORA DO ESCOPO
BigQuery_Sync.gs	Script .gs	`apps_script/integrations/BigQuery_Sync.gs`	Útil operacionalmente, mas não é requisito da integração mobile já criada.	FORA DO ESCOPO

Bibliotecas externas no AppScript: **nenhuma**. Arquivos HTML nesta instalação: **somente Dashboard.html**. Serviço extra a habilitar: **BigQuery API** se você quiser manter a fonte principal de dados da arquitetura atual.

Arquivos reais do repositório

Sem renomear, sem duplicar backend

4. PASSO A PASSO

Criação e cópia dos arquivos no AppScript

PARTE 1

Use os nomes exatos abaixo. No editor do Apps Script, clique em **+**, escolha **Script** para arquivos **.gs** e **HTML** para **Dashboard.html**.

1 Abrir o projeto AppScript correto

Abra o projeto backend que já atende o dashboard web atual. O caminho mais seguro é atualizar esse projeto, não criar um projeto paralelo.

```
Objetivo: manter o mesmo backend para web
+ mobile.
```

2 Criar ou editar `Config.gs`

Crie um arquivo do tipo Script chamado **Config** ou substitua o conteúdo do arquivo existente.

```
Copiar de:
apps_script/utils/Config.gs
```

3 Criar ou editar `Backend_Core.gs`

Esse arquivo precisa conter o **doGet(e)** que detecta **format=json** antes de renderizar o HTML.

```
Copiar de:
apps_script/backend/Backend_Core.gs
```

4 Criar `Mobile_Api.gs`

Crie um arquivo Script chamado **Mobile_Api** e cole o conteúdo integral do arquivo do repositório.

```
Copiar de:
apps_script/backend/Mobile_Api.gs
```

5 Garantir os arquivos de suporte do dashboard

Se você estiver recriando o projeto do zero, copie também os scripts que sustentam o payload da home mobile e do radar.

```
Copiar de:
apps_script/services/Sheet_Readers.gs
apps_script/services/Portfolio_Metrics.gs
apps_script/services/Decision_Engine.gs
apps_script/services/BigQueryService.gs
```

6 Garantir o bloco da Esquilo IA, se desejar

Para o recurso **ai-analysis**, copie os arquivos da IA e das chaves. Sem isso, a home mobile continua possível, mas a leitura sob demanda não fica operacional.

```
Copiar de:
apps_script/services/AI_Service.gs
apps_script/integrations/Api_Keys.gs
```

Não mude os nomes dos arquivos. O Apps Script agrupa tudo no mesmo namespace, então a forma mais segura de refletir a solução entregue é usar exatamente os nomes já existentes no repositório.

5. PASSO A PASSO

Revisões, HTML e configurações obrigatórias

PARTE 2

Esta etapa fecha o projeto AppScript para publicação real: HTML, propriedades, serviço avançado e revisão dos IDs.

7 Criar `Dashboard.html` se quiser preservar o web atual

Crie um arquivo do tipo HTML chamado **Dashboard** e cole o conteúdo do dashboard web existente.

```
Copiar de:
frontend/html/Dashboard.html
```

8 Revisar `APP_CONFIG_` em `Config.gs`

Confirme **spreadsheetId**, **spreadsheetUrl** e os nomes de abas em **sheetNames**. O mobile depende desse mesmo payload.

```
Campos mais críticos:
spreadsheetId
spreadsheetUrl
sheetNames.actions
sheetNames.funds
sheetNames.previdencia
sheetNames.preOrders
sheetNames.aportes
sheetNames.config
sheetNames.dashboardVisual
sheetNames.exportAuxiliar
```

9 Configurar Script Properties

Adicione as propriedades necessárias no projeto Apps Script. Use o painel equivalente de propriedades do script no editor atual.

```
MOBILE_APP_API_TOKEN = opcional, mas
recomendado
GEMINI_API_KEY = necessário para ai-
analysis via Gemini
OPENAI_API_KEY = opcional, fallback da IA
OPENAI_MODEL = opcional
EXTERNAL_MARKET_DATA_ENABLED = opcional
```

10 Habilitar BigQuery API se quiser manter a arquitetura atual

O arquivo **BigQueryService.gs** usa o serviço avançado **BigQuery**. Se ele não estiver habilitado, o backend tende a cair no fallback de planilha onde isso já está tratado.

```
Serviço extra:
Apps Script Advanced Service: BigQuery API
Também habilitar a BigQuery API no projeto
Google Cloud associado, se necessário.
```

6. MAPEAMENTO

De/para dos tipos de arquivo

MAPA RÁPIDO

`CONFIG.GS`, `BACKEND_CORE.GS`, `MOBILE_API.GS`

Criar como **Script (.gs)** no Apps Script e colar o conteúdo dos arquivos correspondentes do repositório.

`DASHBOARD.HTML`

Criar como **HTML**, com nome **Dashboard**, usando o conteúdo de `frontend/html/Dashboard.html`.

`AI_SERVICE.GS` E `API_KEYS.GS`

Criar como **Script (.gs)** apenas se quiser manter o recurso **ai-analysis** ativo.

`BIGQUERY API`

Não é arquivo. É um **serviço avançado** do Apps Script que precisa ser habilitado para o caminho principal via BigQuery.

BIBLIOTECAS

SCRIPT PROPERTIES

Nenhuma. A integração entregue não depende de biblioteca Apps Script externa.

Entram como **configuração manual** do projeto, não como arquivo separado.

Configurações necessárias depois de colar os arquivos

- Revisar **spreadsheetId** e **spreadsheetUrl** em `Config.gs`.
- Revisar os nomes de abas em **APP_CONFIG_.sheetNames**.
- Definir **MOBILE_APP_API_TOKEN** se quiser proteger o acesso externo ao Web App.
- Definir **GEMINI_API_KEY** para a aba de Insights com leitura remota.
- Conferir se **releaseName** e **versionNumber** permanecem como **Pocket Ops / 2.0.0**.

Observação importante

O Flutter atual não exige que você edite um arquivo fixo com a URL do backend. Ele lê a URL no build/execução por **--dart-define**. Isso reduz retrabalho e evita hardcode.

HTML: somente Dashboard

Bibliotecas: nenhuma

7. PUBLICAÇÃO

Como publicar como Web App

DEPLOY

O app Flutter consome a URL final do deploy **/exec**. Sem essa URL publicada, o mobile não conecta.

1 Abrir a publicação

No Apps Script, use o fluxo atual do editor: **Deploy > New deployment** e escolha **Web app**.

2 Executar como

Escolha executar como a conta proprietária do script. Assim o backend acessa a planilha e demais recursos com a mesma credencial do projeto.

3 Acesso para o Flutter

Para teste real no app Flutter, o deploy precisa ser acessível pelo dispositivo. Na prática, o caminho mais simples é acesso externo + **MOBILE_APP_API_TOKEN**.

4 Copiar a URL final

Depois de implantar, copie a URL do tipo **.../exec**. É essa URL que entra no Flutter. Não use a URL **/dev** para o app.

```
Exemplo:
https://script.google.com/macros/s/SEU_DEPLOY/exec
```

Se a URL publicada não puder ser acessada pelo celular, o Flutter vai falhar mesmo que o código esteja correto. Na integração entregue, a URL precisa responder a requisições HTTP feitas pelo app fora do navegador logado.

8. TESTE DOS ENDPOINTS

Validação do backend online

SMOKE TEST

ENDPOINT	EXEMPLO DE URL	O QUE DEVE RETORNAR	COMO SABER SE DEU CERTO
health	`.../exec?format=json&resource=health`	**ok: true**, releaseName, versionNumber e updatedAt	É o teste mais rápido para validar deploy, formato JSON e URL.
dashboard	`.../exec?format=json&resource=dashboard`	Envelope com **data.summary**, **score**, **categorySnapshots**, **updatedAt**	Se vier **ok: true** e o payload tiver a estrutura do dashboard, a home mobile já consegue abrir.
ai-analysis	`.../exec?format=json&resource=ai-analysis`	**data.analysis** com texto da Esquilo IA	Se retornar erro de chave, o deploy está vivo, mas a IA ainda não está configurada.

Se você configurou **MOBILE_APP_API_TOKEN**, acrescente **&token=SEU_TOKEN** às URLs acima.

Onde a URL entra no app mobile

Arquivo que consome a configuração

O app lê a URL e o token em:

```
mobile_app/lib/core/config/app_environment.dart
```

As variáveis usadas são:

```
APP_SCRIPT_BASE_URL
APP_SCRIPT_API_TOKEN
```

Como rodar com a URL publicada

```
flutter run --dart-define=APP_SCRIPT_BASE_URL=https://script.google.com/macros/s/SEU_DEPLOY/exec
```

```
flutter run --dart-define=APP_SCRIPT_BASE_URL=https://script.google.com/macros/s/SEU_DEPLOY/exec --dart-define=APP_SCRIPT_API_TOKEN=seu_token
```

Validação no Flutter: a home deixa o estado de configuração, o cabeçalho mostra o backend ativo, e os blocos de patrimônio, score, alocação, radar e missão passam a renderizar.

10. TROUBLESHOOTING

Problemas comuns

OPERAÇÃO

Abaixo estão os erros mais prováveis ao subir a integração atual, com a leitura direta do que revisar primeiro.

SINTOMA	CAUSA PROVÁVEL	O QUE REVISAR
Erro **Token mobile inválido**	O token passado pelo Flutter ou pela URL não bate com **MOBILE_APP_API_TOKEN**.	Revisar Script Properties e o valor usado em **APP_SCRIPT_API_TOKEN**.
Erro **Recurso mobile não suportado**	O parâmetro **resource** foi enviado com valor diferente de **dashboard**, **ai-analysis** ou **health**.	Revisar a URL chamada ou o serviço Flutter.
HTTP 403 / acesso negado	Deploy sem acesso adequado para o celular ou URL errada.	Revisar a publicação do Web App e confirmar a URL **/exec**.
JSON inválido ou HTML no lugar do JSON	Faltou **format=json** na URL.	Confirmar os parâmetros usados pelo Flutter ou pelo teste manual.
Planilha não localizada	**spreadsheetId** incorreto ou conta sem permissão.	Revisar `Config.gs` e o acesso da conta proprietária do script.
Abas não encontradas	Nomes em **sheetNames** diferentes do que existe na planilha.	Revisar cada chave dentro de **APP_CONFIG_.sheetNames**.
Erro da Esquilo IA	Chave Gemini/OpenAI ausente, inválida ou resposta fora do formato esperado.	Revisar **GEMINI_API_KEY**, **OPENAI_API_KEY**, **Api_Keys.gs** e **AI_Service.gs**.
Erro ao abrir o dashboard web	**Dashboard.html** não foi criado, mas **doGet()** tenta renderizá-lo quando não há **format=json**.	Criar o arquivo HTML com o conteúdo de `frontend/html/Dashboard.html`.

11. CHECKLIST FINAL

Confirmação rápida antes de testar no celular

GO LIVE LOCAL

- Config.gs criado ou atualizado com spreadsheetId e sheetNames corretos.
- Backend_Core.gs atualizado com o desvio para format=json.
- Mobile_Api.gs criado com os recursos dashboard, ai-analysis e health.
- Arquivos de suporte do payload copiados para o mesmo projeto Apps Script.
- Dashboard.html criado se o deploy web atual precisa continuar funcionando.
- Script Properties configuradas, principalmente MOBILE_APP_API_TOKEN e GEMINI_API_KEY.
- BigQuery API habilitada, se você quiser manter a fonte principal da arquitetura atual.
- Web App publicado e URL /exec copiada.
- Teste de health retornando ok: true.
- Teste de dashboard retornando summary, score e categorySnapshots.
- Flutter executado com APP_SCRIPT_BASE_URL apontando para a URL /exec.
- App mobile carregando a home sem ficar preso no estado de configuração.

12. CONCLUSÃO

O que criar ou editar primeiro

RESUMO PRÁTICO

Obrigatório para subir o MVP

- **Config.gs**
- **Backend_Core.gs**
- **Mobile_Api.gs**
- **Deploy Web App com URL /exec**
- **APP_SCRIPT_BASE_URL** no Flutter

Pode ficar para depois

- Ativação da Esquilo IA, se você quiser testar apenas a home primeiro.
- Rotinas fora do escopo mobile como CRUD completo, exportação e sincronização adicional.
- Aprimoramentos de segurança e endurecimento operacional além do token mobile já previsto.

Ordem sugerida de execução: **Config.gs → Backend_Core.gs → Mobile_Api.gs → propriedades → deploy → testes de health/dashboard → Flutter**. Isso já coloca o MVP mobile em condição real de conexão.